

cific settings or how we addressed some of these issues while enabling Geo on the staging environment:

Architecture

We have one Geo secondary node up and running for staging.gitlab.com configured as an all-in-one box with all components colocated on one single node. We are currently not running a Geo HA deployment.

PostgreSQL replication

We decided to use archive recovery to replicate the data from the Geo primary database in the staging environment. The full description of how we have set up it can be found in this runbook at the top section, which refers to single-node installations.

PostgreSQL Foreign Data Wrappers (FDW) - Queries timing-out

GitLab Geo uses PostgreSQL Foreign Data Wrappers (FDW) to perform some cross-database queries between the secondary replica and the tracking database. Despite the technical elegance of this approach, these queries have lead to some problems for Geo.

Originally, the staging environment was still running PostgreSQL 9.6, and Geo benefits from some FDW improvements available only in PostgreSQL 10 and later, such as join push-down and aggregate push-down. This led to some FDW queries timing out during the backfill phase. Since we know this issue is no longer a problem for Geo from PostgreSQL 10 upwards, increasing the statement timeout to 20 minutes on staging was sufficient to allow us to proceed with the backfill.

Since the 13.2 release, we have improved Geo scalability by simplifying backfill operations, eliminating these cross-database queries and removing the FDW requirement.

PostgreSQL version

Staging currently uses PostgreSQL version 11.7. In May 2020, we collaborated with the SRE Datastores team to update the Geo node to use Postgres 11.

Gitaly shards

It is a prerequisite for Geo secondary nodes that they are configured with the same set of logical Gitaly shards. This means that the names in the gitdatadirs config must match those in the Geo primary node.

In staging, most primary Git storage shards are almost empty, and some are experimental artifacts for unfinished features like ZFS storage and Praefect. For this reason, we use only one physical shard and store all logical shards on it for now. To achieve this, each logical Gitaly shard defined in gitdatadirs in the Geo secondary node shares the same path and gitalyaddress.

Secrets

We had some issues with the Geo secondary node in staging while we were using the staging secrets. The Geo secondary node needs to use its own GKMS secrets store, which allowed us to remove conflicting configuration and set secrets for this node, which are different from production.

Deployment

The deploy to the Geo secondary node happens indirectly. One of the final steps while deploying to staging.gitlab.com is to update an attribute in Chef with the version of GitLab that should be running and to set a flag to enable the installation of that specific package. Chef runs roughly every half hour on the Geo secondary node. The next time Chef runs on this node after a successful deployment to staging.gitlab.com, the Geo node will begin upgrading itself. In the worst scenario, the Geo secondary node will take 30 minutes to start the upgrade process.

Known issues

The staging environment does not have the data (repositories, LFS Objects, uploads, etc.) on the file system for every project in the database. To avoid a lot of false-positive errors and a waste of resources trying to resync failed registries over and over again, we decided to enable selective sync at the group level. We currently replicate the gitlab-org group.

There may be some existing replication/verification problems on staging, we aim to track all of them in the Geo Staging Maintenance epic.

Monitoring

- Grafana - Geo Primary insights
- Grafana - Geo Secondary status
- Sentry

Geo Engineer Responsibilities and Support Rotation

Everyone is responsible for QA testing their own MRs on it (whenever applicable). Make sure anything important gets addressed, and anything less is tracked. If you notice a problem with Geo on staging, then feel free to ping/assign the rotation engineer as DRI.

Every month, a Geo backend engineer will be the DRI for monitoring Geo on staging and creating/escalating any issues. This is not an on-call shift and the DRI is not required to fix any issues themselves.

In the last week of their rotation, the outgoing person should set up a meeting with the incoming person to make sure they:

- have SSH access to the staging environment
- can view the Geo admin UI on staging
- have enabled notifications for the geo-staging-gitlabcom project on Sentry
- are familiar with any current ongoing issues with staging geo

The main goals for this rotation:

- Ensure Geo on staging works.
- Distribute responsibility for keeping Geo on staging working.
- Better understand the experience of a customer sysadmin

DRI Daily Tasks

- Check <https://staging.gitlab.com/admin/geo/nodes>. If anything seems weird, ask about it in geo-for-gitlab-dot-com. Cross-post in ggeo if needed. Make sure anything important is addressed.
- Check Sentry. Right now there is a lot of noise, but it can help us identify some edge cases or if something is wrong under the hood.
- Check for triggered alerts. If any are found, follow the documentation for handling them

When Geo on Staging is not Working

- Drive toward a diagnosis by:
 - Investigating with Sentry, Kibana, Grafana, etc, and via SSH into the server.
 - Opening issues
 - Asking others for help
- Help prioritize issues with Engineering Manager, Product Manager, and Infrastructure counterparts.

Rotation Schedule

Month	Name
2023	
December	@jtapiab
November	@aakriti.gupta
October	@brodock
September	@dbalexandre
August	@vsizov
July	@mkozono
June	@ibaum
2022	
July	@dbalexandre
June	@mkozono
May	@cat
April	@mkozono
March	@dbalexandre
February	@cat
January	@vsizov

SECTION: engineering/infrastructure-platforms/tenant-scale/organizations/_index.md

About

The Organizations team's main focus is to develop the Organization entity needed for data sharding and isolation in the context of Cells. The team also offers support for groups, projects, and user profiles within our product.

Contact

To get in touch with us, it's best to create an issue in the relevant project (typically GitLab) and add the ~"group::organizations" label, along with any other appropriate labels.

For urgent items, feel free to use the Slack channel (internal): gorganizations.

Vision

The team is working on developing a more scalable, unified architecture for GitLab through the implementation of Organizations as a new top-level entity.

Organizations serve as logical containers that enable distribution across a cellular architecture while bridging functionality gaps between self-managed and SaaS GitLab instances. The new Organizations entity acts as an umbrella for multiple top-level groups, allowing enterprises to aggregate content across groups, implement organization-wide roles, and isolate content from other Organizations.

Simultaneously, the team aims to improve Groups and Projects by addressing several key challenges: creating more flexible hierarchies to accommodate diverse enterprise structures, reducing confusion around project nesting within groups, enhancing discoverability across the product, standardizing deletion and recovery processes, and improving archiving functionality and visibility.

These improvements collectively work toward creating a more intuitive, flexible system for enterprises to represent their business structure and manage permissions.

Goals

The executive summary goals for the Organizations group include:

- Support GitLab.com's daily-active user growth
- Do not allow a problem with any given data store to affect all users
- Minimize or eliminate complexity for our self-managed use case

Team Members

The following people are permanent members of the Organizations group:

```
{{% team-by-manager-slug manager="mandrewsgl" team="Organizations" %}}
```

Stable Counterparts

The following members of other functional teams are our stable counterparts:

```
{{% engineering/stable-counterparts manager="glopezfernandez" role="Tenant Scale|Principal
```

Engineer, Data Stores|Senior Distinguished Engineer, Ops and Core Platform" %}}

Organization Rollout Counterparts

The following people are supporting our rollout of Organizations.

{{< group-by-slugs atevans>}}

Projects

We are working on different large projects where each project has a Directly Responsible Individual (DRI).

The role of the DRI involves helping to define the scope of the work needed for the project, ensuring clarity on objectives with the responsibility of looking forward 3-6 months to identify potential blocks or risks. Their work is not limited to that area and they also work in other areas as needed.

Project DRI Team
----- ----- -----
Organization backend essentials Tim Chen
Organization isolation: sharding Chen Leonardo da Rosa, Krasimir Angelov
Migrating project/group list pages to Vue Peter Shane
Group archiving Shubham Peter, Abdul
Deletion flow enhancements Remy Abdul, Shane

Meetings

We are a globally distributed group and we communicate mostly asynchronously, however, we also have synchronous meetings. It's unlikely everyone can attend those meetings, so we record them and share written summaries (agenda).

We currently have the following recurring meetings scheduled:

Weekly Wednesday - Organizations Team Sync

- APAC · 04:00 UTC
- EMEA/AMER · 15:00 UTC

SECTION: engineering/infrastructure-platforms/tenant-scale/organizations/process.md

Work

The Product Manager (PM) compiles the list of issues following the product prioritization process, with input from the team, Engineering Manager (EM), and other stakeholders. The iteration cycle lasts until the 2nd Friday of a month, and starts new on the following Monday. Each milestone is identified by the GitLab version set to be released.

Milestone Planning

Before starting a milestone, the group coordinates using planning issues.
We follow this process:

- The PM defines the goals of the milestone.
- The team members comment about the issues they consider relevant for the milestone.
- The PM and EM work together to decide the final list of issues.
- The whole team reviews the items lined up before the milestone begins.

What To Work On

The primary source for things to work on is the planning board,
which lists all issues scheduled for the current cycle.
When you assign yourself to an issue, you indicate that you are working on it.

If anything is blocking you from getting started with an issue immediately,
like unanswered questions or unclear requirements, you can skip it, as long as
you put your findings and questions in the issue.
This helps the next engineer who picks up the issue.

Usually issues are not directly assigned to people, except when
a person has clearly the most knowledge or context to work on an issue.
However, we encourage engineers to have a sense of ownership of specific
projects or epics to make significantly more impact in the company.

Product Development Workflow

We follow the GitLab product development workflow
guidelines. To get a high-level overview of the status of all issues in the
current milestone, check the development workflow board.

The process primarily follows this:

- workflow::ready for design to signal an issue is ready to begin the design work
- workflow::design to indicate that a designer is actively working on an issue
- workflow::planning breakdown to signal that the design is complete and is ready to be broken down into sub-issues for implementation. To preserve context and decisions during the design process, where possible, we re-use the design issue by promoting it to an epic, and add the implementation issues to it. That way, the epic can be used as the SSOT for design, all discussions are in one place, and we don't end up with inconsistencies between the original design issue and the corresponding implementation issue.
- workflow::refinement to signal that an issue needs to be refined by engineering. In this step, an implementation guide and a weight should be added to the issue.
- workflow::ready for development to signal an item is ready to be worked on by engineering

Development Workflow

We follow the GitLab engineering workflow
guidelines. To get a high-level overview of the status of all issues in the

current milestone, check the development workflow board.

As owners of the issues assigned to them, engineers are expected to keep the workflow labels on their issues up to date. When an engineer starts working on an issue, they mark it with the workflow::in dev label as the starting point and continue updating the issue throughout development.

Before closing an issue, it's important to add the workflow::complete label, because this is one

of the requirements for the completed items to appear in the Improvements and Bugs overview of each month's release post. The process primarily follows this diagram:

```
mermaid
graph LR
    classDef workflowLabel fill:#428BCA,color:#fff;
    A(workflow::in dev)::workflowLabel
    B(workflow::in review)::workflowLabel
    C(workflow::verification)::workflowLabel
    F(workflow::complete)::workflowLabel
    A -- Push an MR --> B
    B -- Merged --> C
    C --> D{Works on production?}
    D -- YES --> F
    F --> CLOSE
    D -- NO --> E[New MR]
    E --> A
```

Issue Boards

We track our work on the following issue boards:

- Group::Organizations milestone prioritization
- Group::Organizations cross-functional prioritization
- Group::Organizations planning
- Group::Organizations validation
- Group::Organizations development workflow
- Group::Organizations bugs
- Group::Organizations release posts
- Group::Organizations milestones
- Group::Organizations team members
- Group::Organizations important
- Group::Organizations community contributions

Tracking Dashboards

In addition to issue boards, we also track progress on major initiatives with bespoke dashboards such as:

- Schema migration
- Sharding key migration

The dashboards are part of the Cells Progress Tracker project.

The team has also spun off Epic Dashboards into a project that other teams can use to create their own epic-based tracking dashboards.

Capacity Planning

We use a simple issue weighting system for capacity planning, ensuring a manageable amount of work for each milestone. We consider both the team's throughput and each engineer's upcoming availability from Time Off by Deel using a Google Apps Script.

The weights are intended to be used in aggregate, and what takes one person a certain amount of time may be different for another, depending on their level of knowledge of the issue. We should strive to be accurate, but understand that they are estimates. Change the weight if it is not accurate or if the issue becomes more difficult than originally expected. Leave a comment indicating why the weight was changed and tag the EM and PM so we can better understand the scope and continue to improve.

Weights

To weigh an issue, consider the following important factors:

- Volume of work: expected size of the change to the code base.
- Complexity:
 - Problem understanding: how well the problem is understood.
 - Problem-solving difficulty: the level of difficulty we expect to encounter.

When estimating development work, please assign an issue the appropriate weight:

Weight	Description
1	Trivial
2	Small
3	Medium
4	Large
5	Very Large

Examples

|

| ----- | -----

----- | -----

----- |

| 1: Trivial | The simplest possible change. We are confident there will be no side effects. Negligible complexity.

| Documentation updates, simple regressions, and other bugs that have already been investigated and discussed and can be fixed with a few lines of code, or technical debt that we know exactly how to address, but just haven't found time for yet. |

| 2: Small | A simple change (minimal code changes), where we understand all of the requirements. Some small uncertainties exist but we are confident of a solution.

| Simple features, like a new API endpoint to expose existing data, or regular bugs or

performance issues where all investigation has already taken place.

|

| 3: Medium | A change with a bigger code footprint (e.g. lots of different files, or tests affected). There are uncertainties that we will need to work through.

| Regular features, potentially with a backend and frontend component, or most bugs or performance issues.

|

| 5: Large | A more complex change that will impact multiple areas of the codebase. There may also be some refactoring involved. Requirements are poorly understood and you feel there are multiple important gaps. We will need to break this issue into smaller pieces before we can begin a merge request. | Large features with a backend and frontend component, or bugs or performance issues that have seen some initial investigation but have not yet been reproduced or understood.

|

Anything with a weight of 5 or larger should be broken down if possible.

Backlog Refinement

Every week the engineering team completes a backlog refinement process to review upcoming issues. The goal of this effort is for all issues to have a weight so we can more accurately plan each milestone and also improve our knowledge sharing.

In addition to the backlog refinement process, engineers can estimate any issues without following this backlog refinement process.

Step 1: Identifying Issues for Refinement

The team will identify issues that need to be refined using the workflow::refinement label. If there are issues that are good candidates for the backlog refinement process (without weight, unclear requirements, etc.), please use the label. We will refine a maximum of 5 issues per week.

Refinement issues are auto-generated at the beginning of each week. The scripts can be adjusted in our stage project.

Step 2: Refining Issues

Over the week, each engineer on the team will look at the list of issues selected for backlog refinement. Current backlog refinement issues.

For each issue, team members will review the issues and provide:

- An estimated weight
- A breakdown of issues, if needed
- An implementation guide

When refining issues, consider the following:

- Keep the conversation on the original issues or provide links to relevant discussions in the issue to preserve context
- Update the issue description, implementation plan, and labels as more information is gathered
- For efficiency, engineers can skip the refinement of already refined issues that are marked as completed
- Where the fix is clear and easy, engineers can assign the issue to themselves, give it a weight of 1, push the fix, and close the issue

Step 3: Finalizing Refinement

After engineers have had a chance to provide input, the EM or PM will:

- Assign a weight
- Inform stable counterparts if there are any concerns
- Remove the workflow::refinement label
- Add the workflow::ready for development label

For any issues that were not discussed and given a weight, we will work with the engineers to see if we need to get more information from PM or UX.

Retrospectives

We hold scheduled "per milestone" retrospectives, and can have ad-hoc "per project" retrospectives.

Per Milestone

We have milestone retrospectives issues.

These include the EM, PM, engineers, UX, and all stable counterparts.

Participation is highly encouraged for every milestone. For more information, see group retrospectives created on the 26th of each month, for the currently running milestone.

Per Project

If an issue, a feature, or other sort of project turns into a particularly useful learning experience, we may hold a synchronous or asynchronous retrospective to learn from it. If you think something you're working on deserves a retrospective:

1. Create an issue explaining why you want to have a retrospective and indicate whether this should be synchronous or asynchronous.
1. Include your EM and anyone else who should be involved (like the PM and counterparts).
1. Coordinate a synchronous meeting if applicable. Add all feedback from the retrospective to the issue for future reference.

Error Budgets

GitLab uses error budgets to measure the availability and performance of our features. Each engineering group has its own

budget spend. The current 28-day spend for the Tenant Scale group can be found in this Grafana dashboard.

An error budget exception of 99.85% was approved to allow the group to focus on long-term scalability work.

Dashboards

You can find our group metrics in the Tableau views listed below:

- Top engineering metrics
- Merge request metrics
- Flaky test issues
- Slow RSpec test issues

Engineering Customer/Support Rotation Process

{{< alert type="note" >}}

Note: this process is currently unique to the Groups & Projects half of the team, while Organizations is still in development. We expect to roll out something similar for Organizations when the product is on general release.

{{< /alert >}}

Every two weeks, a Groups & Projects engineer is assigned to be the DRI for the technical assessment of customer support tickets, monitoring gorganizations channel for support issues or master pipeline failures, and addressing bugs and support requests which have been triaged and prioritised.

Process summary

- Every two weeks, a Slack reminder in gorganizationsstandup channel will let the group know that a new support shift is starting for technical assessment triage.
- Every engineer is expected to be aware of their upcoming rotation (as per the schedule below) and take action as per the Slack reminder.
- The DRI currently on rotation should then dedicate their two weeks to the following work (in priority order):
 1. Responding to any alerts for broken master pipeline failures triggered in gorganizations
 1. Working on any scheduled support issues in the current milestone
 1. Working on any scheduled bug issues in the current milestone
 1. Triaging any new issues from the customer support backlog and asking for followup information where needed
- If the DRI is unable to perform an upcoming triage rotation shift due to any reason (e.g. PTO, sick leave, other responsibilities taking precedence), they are expected to swap their rotation with another team member or notify the EM to facilitate. Once the swap is identified, the schedule should be updated.
- The DRI needs to update this issue for the 2025 rotation when handing over to the next participant.

At the end of the rotation, each engineer should provide handover notes inside the team support issue:

- Use the example format in the issue description to summarise what was worked on during the rotation
- Post this directly on the issue, with a ping to the new DRI, in a new root comment (not on a thread)
- If needed, the new DRI should ask clarifying questions on a reply to the comment or in Slack.
- Set up a meeting, if required, to go over more difficult context for handover

Rotation schedule

Schedules are tracked in a Google Sheet (internal link). Please ask before changing this.

SECTION: engineering/management/_index.md

How Engineering Management Works at GitLab

At GitLab, we promote two paths for leadership in Engineering. While there is a healthy degree of overlap between these two ideas, it is helpful and efficient for us to specialize training and responsibility for each of:

- Technical leadership, as represented by Staff and higher-level engineers.
- Professional leadership, as represented by Engineering management.

While technical leadership tends to come naturally to software engineers, professional leadership can be more difficult to master.

This page will serve as a training resource and operational guide for current and future managers.

General leadership principles

All Engineering Managers should follow the general leadership principles set out in the handbook. In particular, it is not uncommon for Engineering Managers to struggle with one or more of the following areas, so we recommend you review them carefully and discuss your confidence with your manager:

- 1-1s
- Providing regular feedback
- Dealing with underperformance
- Engineering manager role explained

Engineering Manager Onboarding

Onboarding is essential for all Engineering Managers at GitLab. As part of onboarding, a Becoming a GitLab Manager issue will be created for each manager when they join, or are promoted. This issue is intended to connect new managers with the crucial information they need, and ensure they have access to all the resources and training available.

Engineering Manager Backup

Each Engineering Manager (EM) is responsible for developing a Backup Plan in the case that an EM has to take an unplanned extended OOO. In order to minimize the team disruptions, this plan should be executed once an EM decides it is the best course of action. An example is below:

| Peer EM Backup (preferably FE / BE pairs) | Senior Team Member | Manager of EM
|
| ----- | ----- | ----- |
| Participate in the Team Retrospective, highlight items to company-wide retro, read items on call if necessary | Work with PM on Backlog Refinement and Milestone Planning | Complete Navan Expense Reports |
| Conduct Synchronous / Asynchronous 1-1's (if more than 1 week) | New Hire Onboarding | Handle Expense Questions |
| Manager Approvals (Access to staging, etc) | | |

Please, consider including timing details. Example: If outage spans last/first week of the milestone, participate in planning the milestone with PM.

When you're done, consider also publishing it to your team page and informing your Peer EM Backup.

Technical Credibility

We expect all managers at GitLab to be technically credible with their teams. Fluency in our core technologies and architectures is essential because it enables managers to participate effectively on technical conversations. In order to maintain this fluency, we encourage managers to participate in coding-related work to an extent. However, please keep the following advice in mind:

- Avoid critical path work. If work has been scheduled for a release or is otherwise blocking other members of the team, it's best left to a developer who can focus on it more holistically. As a manager you should expect regular interruptions to your day that will make you less effective on this kind of task.
- Focus on where you can add the most value. As mentioned above, you won't add value on critical path work, but that doesn't mean you can't add value in other ways. For a great discussion on this, see this article on [How \(and why\) Should Managers Code](#).
- Plan to review more than you write. Following from all of this, most of your "coding-related work" should be in code review - you add more value reviewing code to stay up to date with your team and provide them feedback and guidance. If you find yourself spending more time writing code than you do reviewing it, this may be an indicator that you need to revisit your priorities as a leader.

Management Responsibilities

The sections below aim to inform you of the responsibilities that an engineering manager has at GitLab. It will provide you with the necessary context, information, and process to follow.

- Hiring
- Career Development
- Project Management
- Team Retrospectives
- Development department metrics

Management Roles

The convention at GitLab is to display Manager roles as:

- Manager, Brand Growth Manager in the Marketing Division
- Manager, IT in the Finance Division
- Manager, Software Engineering in the Development Sub-department
- Manager, Support Engineering in the Support Sub-department

Senior Manager roles follow the same convention. For instance

- Senior Manager, Software Engineering in the Development Sub-department
- Senior Manager, Support Engineering in the Support Sub-department

This convention is used in Workday, the system of record. For display in the handbook and to preserve de-facto industry standard role names such as Engineering Manager and abbreviations such as EM, manager roles in the Engineering Division generally follow this naming pattern:

Engineering Manager, [Specialty]

Accounting for temporary management positions, the Senior manager track, different types of manager roles (such as Support), and the potential for one or more specialties yields:

[Acting|Interim] [Senior] [Site Reliability|Support|Quality] Engineering Manager [, Specialty]

Where:

- Acting or Interim roles are temporary management positions.
- Senior manager roles are introduced when needed, usually related to management span of control in the relevant department.
- Some departments have domain specific role names as well as, or instead of, Engineering Manager. Be specific when identifying which manager under Engineering is responsible for certain tasks in order to avoid confusion over the term "EM". For example:
 - Support Operations Manager for Support.
- Specialty - which is maintained in Workday and sync'ed to the handbook - should generally follow these guidelines:
 - Should include a Stage. Choose the primary stage if the manager covers multiple stages.
 - For managers who manage individual contributors, include the group (Stage: Group). Choose the primary group if the manager covers multiple groups.

SECTION: engineering/management/group-retrospectives.md

As a part of our retrospective process, at the end of each release, each Product Group should hold their own Group Retrospective. The goal of the retrospective is to talk through what went well, what went wrong, and what can be improved. Some Engineering sub-departments, such as UX and Quality, also conduct their own retrospectives to feed into the main R&D retrospective and should generally follow the same process outlined here.

Requirements for an efficient retrospective

In order to generate the best results from a retrospective, the following elements must be present:

1. A safe environment for feedback and discussion
1. A plan for advancing discussion from facts to conclusions

Not every retrospective will require both of these elements - for example, an iteration where "nothing went wrong" may see retrospectives that don't benefit from a safe environment. Similarly, an iteration that has nothing but "obvious" problems will probably have a productive retrospective even without a strong plan for the discussion. However, even iterations that seem fine on the surface can harbor deep problems, so it is always best to be prepared.

Establishing a safe environment

Retrospectives are inherently conversations about what went well and what went wrong in a project or iteration. While it's easy to talk about what went well, what went wrong can be a touchy subject. Without a safe environment, issues may go unmentioned, and the group won't improve as rapidly as they otherwise could. To collect as much feedback as possible, we recommend that you observe the following:

1. All retrospectives should have an impartial moderator. This moderator actively attempts to remain objective and is focused on guiding the conversation forward throughout the meeting. If participants feel that the moderator has a stake in the conversation, they may feel as though it is not safe to voice dissent or share concerns.

Normally a manager should act as this moderator. In some circumstances, the moderator may feel a strong need or desire to participate in the conversation. In that case, they should make it clear that their participation is as an individual participant, not as the moderator. If the moderator wants to take a very active role in the discussion, they should find a peer, Director, or other member of the group willing to moderate.

1. Emotions are not only allowed in retrospectives, but they should also be encouraged. Emotional language ("I was angry when...", "It made me sad that...") not only helps convey intensity, it also helps expose issues that may have been difficult to sort out otherwise. Make sure all participants know they are free to express their feelings, although we do expect them to stay consistent with our values.

1. When possible, all parties should be present. No one should have to worry about sharing concerns or experiences because another party isn't there to represent their side of the story. This includes stakeholders outside of the group, where necessary - if someone's role or contributions are going to be discussed, they should have the opportunity to contribute to the retrospective as well.
1. Use an issue to collect feedback asynchronously. Consider using an issue to collect feedback asynchronously. This allows everyone involved to think about and record their feedback on their own time. It is recommended that the group uses the retrospective issue template for the issue.
1. When necessary, get people face to face. After a particularly difficult iteration, or when there's a strong risk that emotions will be running high, it's almost always worth the cost to have everyone in a video meeting to talk through the retrospective in real-time. Some groups choose to not record the meeting if the group prefers to not do so in order to be able to feel that they can speak freely. If the group chooses to do a meeting, consider scheduling it twice with one time good for those in EMEA and Eastern AMER time zones, and one time good for those in APAC and Western AMER time zones. Additionally, some groups choose to not have people leaders in attendance if they are the direct managers of those on the group so that attendees feel they can speak freely.

Having a plan

It's easy for retrospectives to go off the rails if there isn't a good plan for collecting actionable insights. If the moderator doesn't guide the conversation successfully, the retrospective could be dominated by a few "loud voices" or could stay focused on the facts and feelings about the past iteration without drawing any conclusions. To make sure the conversation is productive, we recommend that you have a clear agenda for the conversation. The moderator for the retrospective is then responsible for ensuring the agenda is followed and completed. While the specific nature of this agenda will vary from group to group, we recommend something that follows this general pattern:

1. Introduction - remind people of the purpose of the retrospective, and that the conversation should be scoped to the project/iteration under review. Ensure everyone understands the rest of the agenda.
1. Gather data - don't try to draw any conclusions up front, simply collect facts. This can be done by constructing a timeline, soliciting impressions ("what made you mad, sad, or glad in this iteration?," etc.), or any other number of fact collecting exercises.
1. Generate insights - now that you have all of the facts, try to work together to identify patterns or causal relationships (because we did x, y happened).
1. Decide what to do - with the insights firmly in hand, it should be fairly easy to identify action items for the group. This could be things to change with our process, it could be things to experiment with in the next iteration, or any other number of things. Consider having group members vote on

which things are most important to be acted upon.

1. Close the retrospective - make sure everyone who participated receives positive feedback about how they helped create these results. Observe any surprising or unexpected results from the retrospective, which confirms that the retrospective was valuable. Make sure all of the action items are appropriately assigned to one or more group members with clear expectations for when they should be completed.

Retrospective Notes

Right now we are leaving it to individual Engineering Managers how they would like to collect retrospective notes - GitLab issues, Google docs, etc. After the retrospective is complete, the Engineering Manager has until the Retrospective Summary (shortly after the monthly release) to make relevant notes from the retrospective public. Groups are encouraged to link to these notes from their group page in the handbook.

Learn More

We recommend the following resources if you'd like to learn more about running effective retrospectives:

1. Agile Retrospectives: Making Good Teams Great
1. The Human Side of Post Mortems

SECTION: engineering/management/hiring.md

- Hiring is a priority. GitLab is a fast-growing company, and paying attention to hiring is one of the highest-leverage activities a manager can perform. As long as there are vacancies on your team, hiring should be your top priority.
- Hiring is your responsibility. You may rely on other members of your team to help you evaluate potential candidates. We also have many people at GitLab, most notably in Talent Acquisition, who will help you through the process. But it is always your responsibility to make timely, high-quality hires - speak up if you feel like anything is preventing you from accomplishing this goal.
- Hiring is about making the team better. It is easy to try to find candidates who can merely perform the functions of the role, and more difficult to find people who make the team better while they do it.
- Hiring is hard. It often involves making difficult decisions, and learning to do it well can take years. In times of rapid growth, it may consume 50% or more of your time. Please use every resource available to you, from more experienced hiring managers in the company to the many resources available in our handbook.

SECTION: engineering/management/management-career-development.md

Outside of hiring, the best way to improve the strengths of your team is to practice career development coaching with your developers. Not all team members will want to become Staff Developers or Managers. Instead, identifying their individual career goals and proactively working towards those goals is the most effective way to help team members improve and have ownership over their careers. In addition to the company-wide notes on career mapping and development, here are some important considerations for the Engineering team:

- Technical or Professional Leadership? This is the first section on our career development page for a reason. For Senior Developers, this is the most important question to ask about their career goals, as it will determine where they go next. For anyone who hasn't yet made Senior, this is less important - they need to make Senior first - but it's always a good idea to start talking about this fork in our career paths.
- Assess together. Once you know where your Developer wants to go with their careers, it's time to assess their strengths and weaknesses for the position. This is best done by reviewing the responsibilities and requirements of the job they're interested in. If you both assess it separately, you can compare and contrast your results.
- Create a behavior-based plan. Once you understand the Developer's strengths and weaknesses, help them understand how they can improve by focusing on behaviors they can change. Be careful to avoid tasks like "run this meeting" or "complete this project," and be careful to avoid personality traits like "be less angry" or "be more assertive." "When you realize you're in an argument, take a breath and figure out how to move forward constructively" is an example of a good behavioral action.
- Delegate and give feedback. When you are in agreement about the behavioral action plan, start looking for opportunities to delegate appropriate tasks to the Developer. A Senior looking to move to Staff may benefit from taking some architectural conversations off of your plate, while one looking for a Manager job will learn a lot from tackling some project management initiatives.
- Keep the plan updated. It's important to remember that this is a coaching plan, not a promotion plan. There is never any guarantee that the plan you created will result in a promotion if the Developer "checks all the boxes," and you should feel free to add to or remove from the plan as they grow and you learn more about them. If and when you feel that they are ready for promotion, please follow the normal promotion process.

SECTION: engineering/management/project-management.md

Product is responsible for guiding the direction of our product, and technical leaders are responsible for guiding the technical architecture to meet those requirements. Engineering Managers should participate in both of those conversations. In concert with those conversations, Engineering Managers primary responsibility lies in project management: making sure their team is best positioned to meet both goals as effectively and

efficiently as possible.

- Capacity tracking/planning - while heavy methods of estimation don't fit well with our approach to work, it's your responsibility to keep an eye on the capacity of your team. This allows you to predict capacity, which helps with product planning, and also detect problems earlier if an important schedule is likely to slip.
- Collaboration with stakeholders - in addition to Product Managers and members of your team, you are the primary point of contact for any other stakeholders for the work your team is doing. You should expect to spend time communicating clearly and thoroughly about the state of your ongoing projects.
- Be aware of your team's status. Change can happen quickly - the more you know about what your team is working on and how it's going, the better you'll be able to make the best decisions.
- Balance engineering initiatives - make time for engineering-led initiatives balanced against product's needs for your team. You should also consider your team members' extra responsibilities such as project maintainerships or their involvements in other company wide projects.

SECTION: engineering/monitoring/_index.md

GitLab.com Service Availability

The calculation methodology for GitLab.com Service Availability definition is in the monitoring policy.

More details on definitions of outage, and degradation are on the incident-management page

Historical Service Availability

Year	Month	Availability	Comments
----	-----	----	
2025	May	99.73%	
2025	April	99.97%	
2025	March	100.00%	
2025	February	99.99%	
2025	January	99.98%	
2024	December	99.95%	
2024	November	100.00%	
2024	October	99.66%	
2024	September	99.85%	
2024	August	100.00%	
2024	July	99.99%	
2024	June	99.99%	
2024	May	100.00%	
2024	April	99.96%	
2024	March	100%	
2024	February	99.86%	

2024 January	100%	
2023 December	99.99%	
2023 November	99.99%	
2023 October	99.89	Oct 30 Sev 1
2023 September	99.98%	
2023 August	100%	
2023 July	99.78%	Two severity 1 incidents contributed to ~94% of service disruption.
2023-07-07, 2023-07-14		
2023 June	100%	
2023 May	99.92%	
2023 April	99.98%	
2023 March	99.99%	
2023 February	99.98%	
2023 January	99.80%	
2022 December	100%	
2022 November	99.86%	
2022 October	100%	
2022 September	99.98%	
2022 August	99.92%	
2022 July	99.95%	
2022 June	99.96%	
2022 May	99.99%	
2022 April	99.98%	
2022 March	99.91%	
2022 February	99.87%	
2022 January	99.95%	
2021 December	99.96%	
2021 November	99.71%	
2021 October	99.98%	
2021 September	99.85%	
2021 August	99.86%	
2021 July	99.78%	
2021 June	99.84%	
2021 May	99.85%	does not include manual adjustment for PostgreSQL 12 Upgrade
2021 April	99.98%	
2021 March	99.34%	
2021 February	99.87%	
2021 January	99.88%	
2020 December	99.96%	
2020 November	99.90%	
2020 October	99.74%	
2020 September	99.95%	
2020 August	99.87%	
2020 July	99.81%	
2020 June	99.56%	
2020 May	99.58%	

Related Pages

Production Architecture

GitLab.com Settings
GitLab Performance Monitoring Documentation
Performance of the Application

Related Videos

These videos provide examples of how to quickly identify failures, defects, and problems related to servers, networks, databases, security, and performance.

Monitoring Tools playlist (requires GitLab Unfiltered YouTube account access)
Visualization Tools playlist (requires GitLab Unfiltered YouTube account access)

Monitoring

Pingdom Statistics

We use our apdex based measurements to report official availability (see above). However, we also have some public pingdom tests for a representative view of overall performance of GitLab.com. These are available at <https://stats.pingdom.com>. Specifically, this has the availability and latency of reaching

a GitLab.com issue. For reference, it is the first gitlab-ce issue.
GitLab.com "plain and simple" called the GitLab public check.

Monitoring Infrastructure

We use Grafana Mimir to ingest and query metrics. Mimir is an open-source, distributed time series database that extends Prometheus. You can read more about its implementation in our Runbook Docs.

Monitoring Dashboards

Metrics can be viewed in Grafana. The Grafana Explore dashboard allows querying of all data in Mimir using PromQL.

Access requires a @gitlab.com email address through Google SSO
Highly Available setup
Alerting feeds from this setup
Separated from the public for compliance, security and availability reasons

Adding Dashboards

To learn how to set up a new graph or dashboard using Grafana, take a look at the following resources:

Guide to setting up Grafana dashboards by Grafana
YouTube video showing how to set up a dashboard
The Grafana repo where we keep an archive of InfluxDB dashboards created in Grafana. Use these to see details in the file structure, but note that the repo is truly an archive (nothing populates from it) and can be out of date.

Need access to add a dashboard? Ask any team lead within the infrastructure team.

Dashboards for stage groups

We have a set of monitoring dashboards designed for each stage group. These dashboards are designed to give an insight, to everyone working in a feature category, into how their code operates at GitLab.com scale. They are grouped per stage group to show the impact of feature/code changes, deployments, and feature-flag toggles.

1. List of dashboards for each stage group (GitLab team members only).

1. Guide to getting started with dashboards for stage groups

1. YouTube video introducing the stage group dashboards

The dashboards for stage groups are at a very early stage. All contributions are welcome. If you have any questions or suggestions, please submit an issue in the Scalability Team issues tracker.

Selection of Useful Dashboards from the Monitoring

Blackbox Monitoring

GitLab Web Status: front end perspective of GitLab. Useful to understand how GitLab.com looks from the user perspective. Use this graph to quickly troubleshoot what part of GitLab is slow.

GitLab Git Status: front end perspective of GitLab ssh access.

Private Whitebox Monitor

Host Stats: useful to dive deep into a specific host to understand what is going on with it. Select a host from the dropdown on the top.

Business Stats: shows many pushes, new repos and CI builds.

Daily overview: shows endpoints with amount of calls and performance metrics. Useful to understand what is slow generally.

Logs

Network, System, and Application logs are processed, stored, and searched using the ELK stack. We use a managed Elasticsearch cluster on GCP and as such our only interface to this is through APIs, Kibana and the elastic.co web UI. For monitoring system performance and metrics, Elastic's x-pack monitoring metrics are used. They are sent to a dedicated monitoring cluster. Long-term we intend to switch to Prometheus and Grafana as the preferred interface. As it is managed by Elastic they run the VMs and we do not have access to them. However, for investigating errors and incidents, raw logs are available via Kibana.

Staging logs are available via a separate Kibana instance.

Kibana dashboards are used to monitor application activity, spam events, transient errors, system and network authentication events, security events, etc. Commonly used dashboards are the Abuse, SSH, and Rack Attack dashboards.

One can view how we log our infrastructure as outlined by our runbook.

The policy related to log management can be found in [the monitoring policy].

Adding dashboards

To learn how to create Kibana dashboards use the following resources:

Kibana Dashboard tutorial from Elastic.com
Building a dashboard
Using TimeLion for time series visualization

GitLab Profiling

Go services

Stackdriver Continuous Go Profiling can be used to have a better understanding of how our Go services perform and consume resources. (requires membership of the Google Workspace stackdriver-profiler-sg group)

It provides a simple UI on GCP with CPU and Memory usage data for:

Gitaly (and Praefect)
Workhorse
GitLab Pages
GitLab Container Registry

For more information, there's a quick video tutorial available.

We also did a series of deep dives by pairing with the development teams for each project in this issue, this resulted in the following videos:

Gitaly
GitLab Pages
GitLab Registry

Instrumenting Ruby to Monitor Performance

Blocks of Ruby code can be "instrumented" to measure performance.

Documentation of instrumentation with more detail on how to implement this
An example of how this is used for GitLab itself, can be found in this initializer.

Other Tools

Sentry

Error tracking service.

Documentation

How to investigate a 500 error - Sentry / Kibana Demo

Diagnose Errors on GitLab.com - Searching Sentry

Setting sentry alerts for your group

Creating alert rules allows groups to monitor their features and help catch issues proactively. This helps in getting the issues fixed before they breach the error budget SLO which in turn helps in keeping the GitLab.com Service Availability high.

Steps for creating the alerts:

1. Visit Sentry's alert rules dashboard.

1. Click on "Create Alert" button at the top right.

1. Set the required conditions as per your group's feature categories.

1. Create a new public slack channel with the following naming convention "ggrouppnamealerts".

Eg: ggoverncompliancealerts

1. Select this channel for sending the alert notifications.

1. Monitor the group for any new alerts and work towards resolving those.

Sitespeed.io

Tool that helps you monitor, analyze and optimize your website speed and performance.

Documentation

GitLab.com Sitespeed Measurement Repository

How we used sitespeed.io to measure Frontend performance

SECTION: engineering/monitoring/staging-monitoring.md

Staging environment has the same service-level monitoring rules as Production:

> A service is considered available when:

>

> 1. The Apdex score of the service is above its Service Level Objective (SLO),

> 1. AND The Error rate is below its Service Level Objective (SLO).

The goal of Staging Monitoring is to have SLO alerts that can be used to halt an ongoing deployment. This would allow failing fast and catching bad deployments before they reach Production. To enable this we need to ensure that the environment has enough base-load traffic and the signal of an SLO failure is strong.

Staging environment doesn't have as much user activity as Production since it doesn't have the same amount of real users. The environment is mostly used by test automation like GitLab QA pipelines and engineers who may test their code manually. These activities don't generate enough traffic so a custom load emulation tool was designed to create artificial traffic to compensate for the lack of signal from real users.

Load emulation

CMBR is a web-crawler that generates traffic on a target GitLab instance. It can generate traffic for these services: Web, API, Git, Registry and Pages.

Known limitations:

- Crawler only runs GET requests and doesn't create data. However, this problem is not as critical as existing E2E tests that are being run against Staging cover GitLab basic functionality.
- The load is synthetic and doesn't cover all use cases, especially the edge cases that rely on specific data.
- Existing Staging test data is different from Production.
- Rate limits such as IP or user rate limits may affect traffic generation.
- Staging environment has low specs in comparison to Production. Load needs to be tuned so that it doesn't break the environment.
- Git traffic is HTTP-only at the moment.

Crawler configuration on Staging

CMBR generates traffic for Staging using scheduled pipelines for gstg and cny-gstg. The tool uses a dedicated user with auditor role to generate load and bypass rate limits (Credentials for Staging Crawler stored in 1Password Engineering vault).

Throughput is controlled by environment variables and tuned differently for each service. It's worth noting that if load needs to be increased, Staging environment's performance should be considered and verified that it doesn't affect existing GitLab QA pipelines. Otherwise, it may bring intermittent errors in the test runs and affect the deployment.

Staging Service Monitoring

Alertmanager sends SLO alerts to feedalertsstaging Slack channel if the Apdex score or Error rate is violated for specific service. Then Infrastructure team investigates the alert further.

To view specific service health the below dashboards can be used:

- GitLab Web overview
- GitLab API overview
- GitLab Git overview
- GitLab Registry overview
- GitLab Pages overview

More detailed information about service monitoring can be found on [Monitoring of GitLab.com](#) page.

If you have any questions about the current setup, please reach out to [fstagingservicelevelmonitoring](#) Slack channel.

Ongoing Work

- Improving the quality of Staging SLO alerts - epic668
- Halt deployment to Production if Staging SLI degrades - epic771
- Generate SSH traffic using CMBR - issue5

SECTION: engineering/open-source/_index.md

We believe in Open Source

As a company, GitLab is dedicated to open source. Not only do we believe in it, but we use it, and we give back to it. Not just through GitLab, but through contributions to other open source projects.

The purpose of this page is to document how a GitLab employee can:

- Create an open source project on behalf of GitLab
- Contribute to a third-party open source project on behalf of GitLab
- Use a third-party open source code in a GitLab's project

Growth Strategy

As an open source project, we want to stay healthy and be open for growth, but also ready to accommodate a 10x factor of our community. In order to achieve that, we've outlined a strategy that is a collaboration between multiple departments.

General notes

- GitLab Code of Business Conduct and Ethics
- PIAA

Creating an open source project

See Creating a new project for the instructions.

Contributing to a third-party project on behalf of GitLab

Contributor License Agreements (CLAs)

If you're contributing to an open source project on behalf of GitLab, you may be required to enter into a CLA.

- In accordance with the Authorization Matrix Policy, Legal approval is required in order for you to enter into a CLA on behalf of GitLab.
- Visit the Legal & Corporate Affairs Product handbook for further information on this process.

Contributing to a project on GitHub

If your GitHub account's primary email is not your @gitlab.com email, you can add it as an

additional address. No need to create a separate account.

1. Fork the repository you want to contribute to into your account
2. Follow the usual pull request flow.

In the future, we might have a single organization for forks. That will allow us to track various metrics about contributions made by GitLab employees.

If a project does not have a license

GitLab allows contribution to open source licensed projects. If the project to you which you wish to contribute does not have a license:

1. Hold off making any contributions to the project until a license is in place.
1. Open an issue in the project (or contact the maintainers through other documented means).
1. Request the project adds an appropriate license citing an inability to contribute to unlicensed code.
1. Potentially refer to licenses used in other projects in the same group/organization as an easy reference for the maintainer.
1. If an open source license is added, you may then contribute to the project.

Using open source software

The guidance for using open source software, including details of which licenses are pre-approved for use, is now available in the Legal & Corporate Affairs handbook.

Using forks in your code

Avoid using forked code and try to contribute your change upstream.

It's typical for forks to fall far behind the upstream repository and such dependencies become a source of pain:

- Rebasing the branch may become non-trivial and it'd become hard to bring such dependency up to date.
- Some other library in your project might depend on the original version, creating a diamond dependency problem.

There may be good reasons to create a fork:

- To fix a security issue that is not being fixed upstream fast enough if it's affecting us or our customers
- Any other reasons? Talk to your peers and use your best judgement.

If you decide to create a fork, make sure you open an issue that:

- Describes the reason for the fork to exist
- Links to the MR(s) where the fork was introduced as a dependency
- Links to any relevant issues in the upstream project. If the issue was not reported already, make sure you report it in the project's issue tracker. This is important because if the

project's maintainers don't know about it they will not fix it

- Links to any opened MRs/PRs to fix the issue upstream
- Describes the remediation work needed to start using the upstream code again
- If it's not just changes in the forked code, but also some modifications in your code to use the fork, consider putting a TODO and a link to this issue next to that code in a comment

GPL Cooperation Commitment

Visit the Legal & Corporate Affairs Product handbook for further information on this.

SECTION: engineering/open-source/growth-strategy.md

Strategy

We will be executing in 5 key areas in support of the company's broader dual-flywheels strategy to attain more contributions. The 5 key areas together are the building blocks of our contributor & contribution 10x acceleration strategy.

mermaid

flowchart LR

```
subgraph moreContributions["More Contributions"]
    contributorIncrease["Contributor Increase"]
    contributionIncrease["Contribution Increase"]
    bothIncrease["Contributor & Contribution Increase"]
    increaseValue["Increase Contribution Value"]
    improveJourney["Improve Contributor Journey"]
    fosterDiversity["Foster Diversity, Equity, and Inclusion"]
    scaleCommunity["Scale the Community"]
    expandOutreach["Expand Outreach"]
    scaleCommunity-->improveJourney
    scaleCommunity-->increaseValue
    scaleCommunity-->fosterDiversity
    expandOutreach-->scaleCommunity
    increaseValue-->contributionIncrease
    improveJourney-->contributorIncrease
    fosterDiversity-->bothIncrease
end
style moreContributions fill:FFF, stroke:9370DB, stroke-dasharray: 5 5
style contributionIncrease fill:9370DB,stroke:9370DB,stroke-width:10px
style contributorIncrease fill:9370DB,stroke:9370DB,stroke-width:10px
style bothIncrease fill:9370DB,stroke:9370DB,stroke-width:10px
style improveJourney color:6b4fbb, stroke:9370DB
style increaseValue color:6b4fbb, stroke:9370DB
style fosterDiversity color:6b4fbb, stroke:9370DB
style expandOutreach color:6b4fbb, stroke:9370DB
style scaleCommunity color:6b4fbb, stroke:9370DB
click improveJourney "./improve-contributor-journey" self
click increaseValue "./increase-contribution-value" self
```

click fosterDiversity "/foster-diversity-equity-and-inclusion" self
click expandOutreach "/expand-outreach" self
click scaleCommunity "/scale-the-community" self

Current focus

We're prioritizing these 5 key items to maximize our impact within our current capacity. While all initiatives are valuable, this focused approach allows us to drive meaningful change more effectively.

Reduce Open Community MR Age
Create a compelling contributor value proposition
Contributor advancement system
Returning & Frequent Contributors
Non-code contributions

Improve Contributor Journey

Provide an outstanding, highly efficient and fast contributor experience, from onboarding to getting a change merged. One of the tactics is to reduce contribution stumbling blocks to make the contributor journey more efficient. These stumbling blocks will be identified through gathering feedback from wider community contributors, product teams and GitLab Team Members on contribution friction.

Reduce Open Community MR Age

Why: Improve the speed of contribution to production by reducing Open community MR Age (OCMA) & review time. We have identified product groups with the highest OCMA. Analysis and improvements are needed to address product groups with the biggest opportunity. In addition to improving MR review and gathering feedback.

DRI: Contributor Success team

Product & engineering alignment

Why: Within product groups, contribution submissions, backlog, and technology stack vary. Healthy community backlog alignment and the establishment of a common best practice for outreach are vital for contributor success. In addition, there should be a unified & known workflow for Wider Community Contributions.

DRI: Product Operations, Developer Advocacy team & Contributor Success team

Simplify & improve contribution guides

Why: Make contribution guides easy to navigate. Our current contribution guides are fragmented and can be hard for new contributors to navigate and understand.

DRI: Developer Relations team

Improve contribution tooling

Why: Provide fast and efficient contributor experience via our tooling. Our contributor

tooling needs to be optimized for contributor productivity

Epic: <<https://gitlab.com/groups/gitlab-com/quality/-/epics/2>>

DRI: Developer Tooling team

Increase Contribution Value

Incentivize, attract and retain contributors by providing a compelling value and regular recognition of contributors for their work. Contributor career advancement materials and awards.

Create a compelling contributor value proposition

Why: We need a clear definition of what drives people to contribute to GitLab and to present a compelling value proposition for increasing code contribution. One of the programs to drive this challenge is our Co-Create program for customers.

DRI: Developer Relations team

Contributor recognition & advancement system

Why: In open source projects, contributors are motivated not only by solving bugs or adding features but also by gaining experience and building their online presence. This can be solved through